GLOBAL LOGISTICS MANAGEMENT SYSTEM

GLMS

Part 1: Enterprise Analysis and Architecture Report

TechMove Logistics | Chief Solutions Architect

Document Information	Details
Client	TechMove Logistics
Project	Global Logistics Management System
Document Type	Enterprise Architecture Report
Framework	TOGAF ADM (Phases A-D)
Version	1.0

1. Introduction

TechMove Logistics currently operates through a legacy environment of disconnected spreadsheets, manual telephone coordination, and unstructured email chains. This approach has produced systemic problems including data fragmentation, missing invoices, repeated compliance failures related to expired contracts, and significant bottlenecks in service delivery. The business case for modernisation is clear and urgent.

This report presents the architectural blueprint for the Global Logistics Management System (GLMS), a proposed enterprise-grade platform designed to centralise contract management, validate and process service requests, and integrate international financial data through external APIs. The document serves as the primary technical reference for the TechMove Board of Directors prior to any development activity commencing.

Enterprise architecture frameworks exist to bridge the gap between strategic business goals and the technical systems that support them. *(The Open Group, 2022)*

This report applies TOGAF as the governing framework and identifies three Gang of Four (GoF) design patterns that will structure the GLMS codebase. It further defines the non-functional requirements of availability and interoperability, and articulates a scalability strategy capable of absorbing sudden large-scale growth.

2. Enterprise Architecture Framework: TOGAF

The TOGAF Architecture Development Method (ADM) has been selected as the enterprise architecture framework for this project. TOGAF is the most widely adopted enterprise architecture framework globally and provides a structured, iterative approach to designing, planning, implementing, and governing enterprise information architecture.

TOGAF ADM provides a reliable, practical method for developing enterprise architectures, with phases that can be adapted to meet specific project contexts and organisational needs. *(The Open Group, 2022)*

The ADM cycle will be applied across four key phases for the GLMS project. Each phase produces concrete deliverables that directly inform the next stage of development.

2.1 Phase A: Architecture Vision

Phase A establishes the strategic intent and obtains stakeholder commitment before detailed architectural work begins. For TechMove, the architecture vision articulates the transformation from a fragmented, manual-process environment to a centralised, automated, and contract-driven logistics platform.

Key stakeholders identified at this phase include logistics managers responsible for raising service requests, client organisations bound by contractual SLAs, the finance team managing international invoice settlement, and system administrators responsible for infrastructure operations. The primary driver is the elimination of compliance failures caused by expired or mismanaged contracts, which represents a direct regulatory and financial risk to the organisation.

The architecture vision deliverable for this project commits to the following outcomes: a single source of truth for all contract and client data, automated status lifecycle management, validated service request workflows, and real-time currency conversion for international invoicing.

2.2 Phase B: Business Architecture

Phase B maps the current-state business processes against the target-state processes. The current state at TechMove is characterised by manual contract entry via spreadsheets, no automated status tracking, phone-based service request coordination with no audit trail, and invoice reconciliation performed manually across disconnected email threads.

The target-state business architecture introduces automated contract lifecycle management with status transitions governed by system rules (Draft, Active, Expired, On Hold), a validated service request workflow that enforces active-contract linkage before any request can be submitted, and a financial integration layer that automatically converts invoice amounts using live external exchange rate data.

Business architecture in TOGAF defines the business strategy, governance, organisation, and key business processes to identify how they support the overall goals of the enterprise. *(The Open Group, 2022)*

2.3 Phase C: Information Systems Architecture

Phase C defines both the data architecture and the application architecture. The data architecture identifies the core entities and their relationships. The primary entities within the GLMS domain are Clients, Contracts, Service Requests, Invoices, and SLA Definitions.

Contracts maintain a foreign key reference to a Client, enforcing that every contract is owned by an identifiable client. Service Requests maintain a foreign key reference to a Contract, enforcing the business rule that no request may be raised against an inactive or non-existent contract. Invoices link to Service Requests and carry both a base amount and a currency code, enabling the financial integration layer to resolve conversion at runtime.

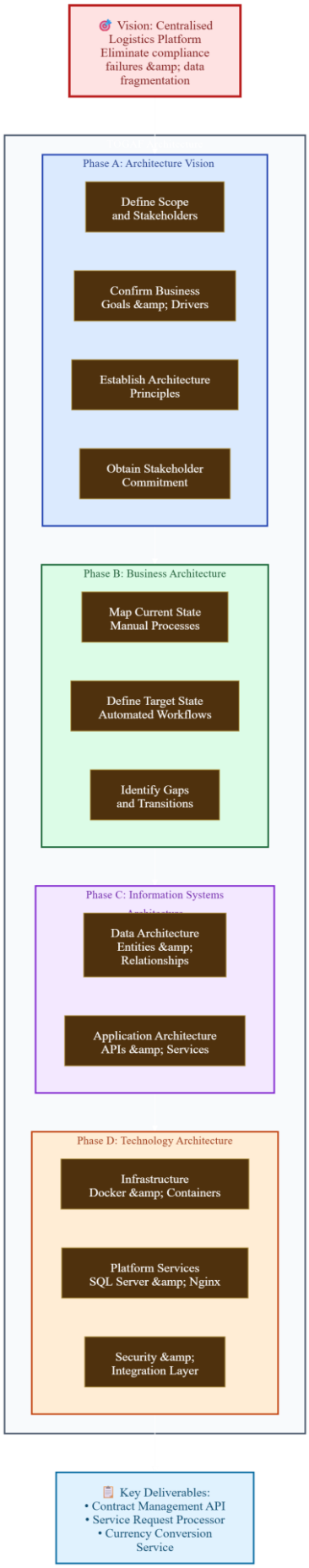
The application architecture identifies three discrete service modules: the Contract Management API, which handles CRUD operations and status transitions; the Service Request API, which performs contract validation before persisting any new request; and the Currency Conversion Service, which consumes an external exchange rate API and supplies converted financial values to the invoice layer.

2.4 Phase D: Technology Architecture

Phase D specifies the infrastructure and technology stack. The GLMS will be built on C# with ASP.NET Core as the API framework. Data persistence will be managed through Entity Framework Core connected to a SQL Server relational database. The system will be containerised using Docker, with Docker Compose orchestrating the API service and the database service as separate containers communicating over an internal bridge network.

An Nginx reverse proxy will sit in front of the application containers to handle request routing, SSL termination, and load distribution across multiple API container instances. External API calls for exchange rate data will be made over HTTPS using a typed HttpClient registered with the ASP.NET Core dependency injection container. The technology architecture is designed from the outset to support horizontal scaling without structural changes to the application code.

TOGAF ADM High-Level Visual (Professional Blue/Gray Scheme)


🎯 Vision: Centralised Logistics Platform Eliminate compliance failures & data fragmentation
Phase A: Architecture Vision
Define Scope and Stakeholders
Confirm Business Goals & Drivers
Establish Architecture Principles
Obtain Stakeholder Commitment
Phase B: Business Architecture
Map Current State Manual Processes
Define Target State Automated Workflows
Identify Gaps and Transitions
Phase C: Information Systems
Data Architecture Entities & Relationships
Application Architecture APIs & Services
Phase D: Technology Architecture
Infrastructure Docker & Containers
Platform Services SQL Server & Nginx
Security & Integration Layer
📋 Key Deliverables:
• Contract Management API
• Service Request Processor
• Currency Conversion Service

3. GoF Design Pattern Selection

Design patterns represent proven solutions to recurring design problems in object-oriented software. The Gang of Four catalogue of 23 patterns remains the foundational reference for structured, maintainable software design. *(Gamma et al., 1994)*

Three patterns have been selected for the GLMS: Observer, Strategy, and Decorator. These three patterns address fundamentally different design concerns and together cover the most complex areas of the GLMS domain. Each selection is justified below, followed by a UML class diagram showing the integrated structure.

3.1 Observer Pattern (Behavioural)

The Observer pattern defines a one-to-many dependency between objects such that when one object changes state, all its dependents are notified and updated automatically. In the GLMS, the Contract entity is the subject. When a contract transitions from Active to Expired, multiple downstream systems must react: the notification service must alert the relevant account manager, the audit logger must record the event with a timestamp, and the service request validator must invalidate any pending requests linked to that contract.

The Observer pattern is appropriate whenever a change to one object requires changing others, and you do not know how many objects need to change. *(Gamma et al., 1994)*

Without the Observer pattern, the Contract class would need to directly reference and invoke all three downstream services, creating tight coupling and making the system difficult to extend. The pattern ensures that the contract entity remains isolated from its consumers. New observers, such as a billing service or an SLA metrics tracker, can be registered without modifying the Contract class.

3.2 Strategy Pattern (Behavioural)

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It allows the algorithm to vary independently from the clients that use it. In the GLMS, the Service Request validation logic is the primary candidate. Different contract types may

require different validation rules. A standard freight contract may require only that the contract status is Active, while a premium SLA contract may additionally require that the requesting party has no outstanding invoice debt and that the requested service type falls within the agreed scope.

The Strategy pattern is useful when you want to define a class that will have one behaviour that is similar to other behaviours in a list. *(Gamma et al., 1994)*

By encapsulating each validation algorithm behind a shared IValidationStrategy interface, the ServiceRequestProcessor can accept any strategy at runtime without modification. This makes it straightforward to introduce new contract categories in the future, satisfying the open/closed principle of SOLID design.

3.3 Decorator Pattern (Structural)

The Decorator pattern attaches additional responsibilities to an object dynamically. It provides a flexible alternative to subclassing for extending functionality. In the GLMS, the CurrencyConversionService is the target. The base implementation makes a live HTTP call to an external exchange rate API. However, calling this API on every invoice conversion would introduce unnecessary latency and create a dependency on external availability.

A CachedCurrencyServiceDecorator wraps the base service and intercepts conversion requests. If a rate for the requested currency pair has been retrieved within the last hour, the decorator returns the cached value without making an external call. If the cache has expired or the rate is not present, the decorator delegates to the underlying service and stores the result. This is entirely transparent to the consuming code, which depends only on the ICurrencyService interface.

The Decorator pattern provides a flexible way to add responsibilities to objects without resorting to subclassing. It is especially useful when you want to add features to individual objects rather than to an entire class hierarchy. *(Gamma et al., 1994)*

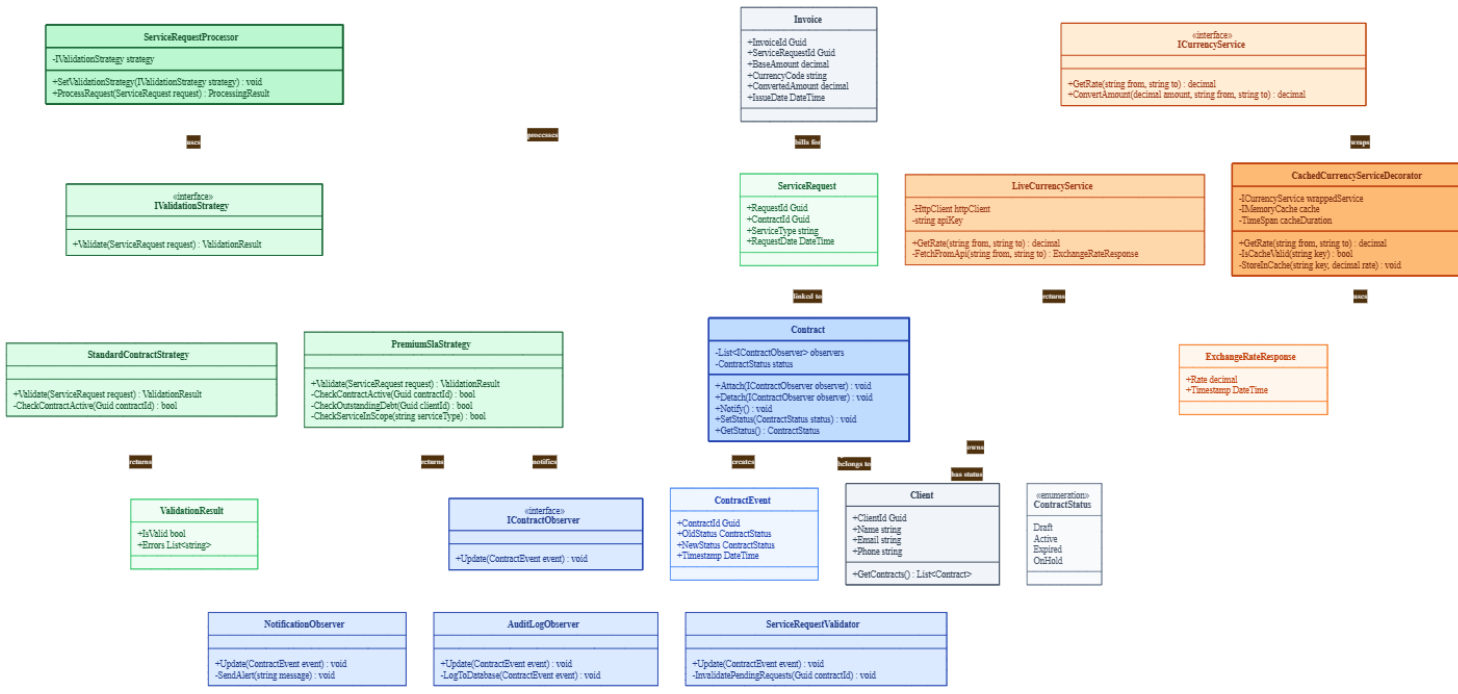
3.4 UML Class Diagram

The following UML class diagram illustrates the integrated structure of the three selected patterns as they will be implemented in C# code. The diagram shows the relationships between interfaces, concrete implementations, and the domain entities that bind the patterns together within the GLMS architecture.

Pattern	Key Interface / Abstract	Concrete Implementations
Observer	IContractObserver: Update(ContractEvent)	NotificationObserver, AuditLogObserver, ServiceRequestValidator
Strategy	IValidationStrategy: Validate(ServiceRequest)	StandardContractStrategy, PremiumSlaStrategy
Decorator	ICurrencyService: GetRate(from, to)	LiveCurrencyService, CachedCurrencyServiceDecorator

Note: A visual UML diagram in standard notation is included as a companion artefact. The table above serves as a structural reference for this document. In the class diagram, Contract implements the Subject role of the Observer pattern and holds a List<IContractObserver>. The ServiceRequestProcessor holds a reference to IValidationStrategy injected at construction. The CachedCurrencyServiceDecorator holds a private reference to ICurrencyService, fulfilling the wrapping relationship characteristic of the Decorator pattern.

UML Class Diagram (Clean Monochromatic with Pattern Highlighting)



4. Non-Functional Requirements

Non-functional requirements define the quality attributes that a system must exhibit and are often more critical to enterprise system success than individual functional features. *(Bass, Clements and Kazman, 2021)*

4.1 Availability

Availability is defined as the proportion of time a system is operational and accessible to its intended users. For TechMove Logistics, the GLMS handles time-sensitive operations: contracts must be accessible to both internal managers and external clients at any time, and service requests may be raised around the clock given TechMove's global client base across multiple time zones.

The GLMS must achieve a minimum availability target of 99.5%, equivalent to no more than approximately 43.8 hours of unplanned downtime per year. Achieving this requires a multi-layered approach to resilience. At the infrastructure layer, Docker containers must be run with health checks and automatic restart policies so that any container failure results in an automatic recovery without manual intervention. At the application layer, all database operations must implement retry logic with exponential backoff to handle transient connection failures gracefully.

Planned maintenance windows must be scheduled outside of peak business hours, and a blue-green deployment strategy should be adopted so that new application versions can be deployed without any user-facing downtime. This means maintaining two identical production environments and switching traffic between them during releases rather than taking the live environment offline.

4.2 Interoperability

Interoperability is defined as the ability of the GLMS to exchange data and function correctly with external systems, third-party services, and future internal platforms that have not yet been defined. This is a critical requirement given that the GLMS must consume an external exchange rate API for currency conversion and must be designed for future integration with accounting platforms, customs declaration systems, and client-facing portals.

Interoperability in enterprise systems requires that interfaces be defined using open standards so that integration partners are not required to adopt proprietary protocols. *(Fowler, 2002)*

The GLMS API will expose all endpoints using RESTful conventions over HTTPS, with request and response payloads serialised as JSON. All date fields will use the ISO 8601 standard (YYYY-MM-DD) and all currency amounts will carry an explicit ISO 4217 currency code alongside the numeric value. This ensures that any third-party system capable of making standard HTTP requests can integrate with the GLMS without custom protocol negotiation.

The external currency conversion dependency will be encapsulated behind the ICurrencyService interface, meaning that if the chosen external API is replaced or upgraded in the future, only the concrete implementation of that interface needs to change. All consuming code within the GLMS will continue to work without modification. API versioning will be implemented using URI path versioning (for example /api/v1/contracts) so that breaking changes in future versions do not disrupt existing integrations.

5. Scalability Strategy

Scalability is the capability of a system to handle a growing amount of work by adding resources to the system. Enterprise systems must plan for growth that can occur both gradually and suddenly. *(Abbott and Fisher, 2015)*

The scenario presented is that TechMove acquires a competitor and must absorb an additional 50,000 contracts overnight. This represents a sudden, large-scale increase in data volume, API traffic, and concurrent user sessions. The GLMS architecture must be capable of handling this load without a full redesign. Three specific strategies are employed to address this: horizontal scaling, load balancing, and caching.

5.1 Horizontal Scaling

Horizontal scaling refers to the practice of adding additional instances of a service rather than increasing the resources of a single instance. The GLMS API is designed as a stateless service from the outset, meaning that no session state is stored within the application process. Authentication is handled using JWT tokens carried in the request header, so any API instance can handle any request without needing to consult a shared session store.

Because the API is stateless, adding additional Docker containers running the GLMS API image is sufficient to increase throughput. Docker Compose supports scaling individual services with a simple configuration parameter. In a cloud-hosted environment, this can be automated using container orchestration platforms such as Kubernetes, which can monitor CPU and memory utilisation and automatically provision or deprovision instances based on observed demand. If TechMove adds 50,000 contracts overnight and query traffic increases proportionally, additional API instances can be provisioned within minutes without any downtime to the existing service.

Horizontal scaling through stateless service design is one of the most reliable approaches to handling unpredictable load, as it allows capacity to be matched to demand incrementally rather than requiring over-provisioning at the outset. *(Abbott and Fisher, 2015)*

5.2 Load Balancing

A load balancer distributes incoming HTTP requests across all available API instances. In the GLMS architecture, Nginx is deployed as a reverse proxy and load balancer in front of the API

container cluster. Nginx uses a round-robin distribution algorithm by default, which ensures that no single API instance receives a disproportionate share of traffic under normal operating conditions.

The load balancer also performs health checking on each upstream API instance. If an instance fails to respond within a defined timeout, Nginx automatically removes it from the active pool and redirects traffic to the remaining healthy instances. This ensures that a single container failure does not result in a visible outage to end users. The load balancer itself can be made highly available by running in an active-passive configuration with automatic failover, eliminating it as a single point of failure.

5.3 Caching

Caching reduces the number of computationally expensive operations by storing the results of previous operations and serving them directly for subsequent identical requests. In the GLMS, two levels of caching are applied.

At the application layer, the CachedCurrencyServiceDecorator, described in Section 3.3, stores exchange rates in memory with a time-to-live of 60 minutes. Currency rates do not change second by second, so this cache eliminates the majority of outbound API calls to the exchange rate provider without meaningfully reducing data freshness for invoicing purposes.

At the database layer, the sudden addition of 50,000 contracts means that common read operations, such as listing all active contracts for a given client, will be executed by many concurrent users simultaneously. A distributed cache such as Redis can be introduced to store the results of frequent, expensive database queries. When a logistics manager requests the list of active contracts for a client, the API first checks the Redis cache. If the result is present and within its expiry window, it is returned immediately without a database query. If not, the query is executed, the result is cached for future requests, and the response is returned to the user.

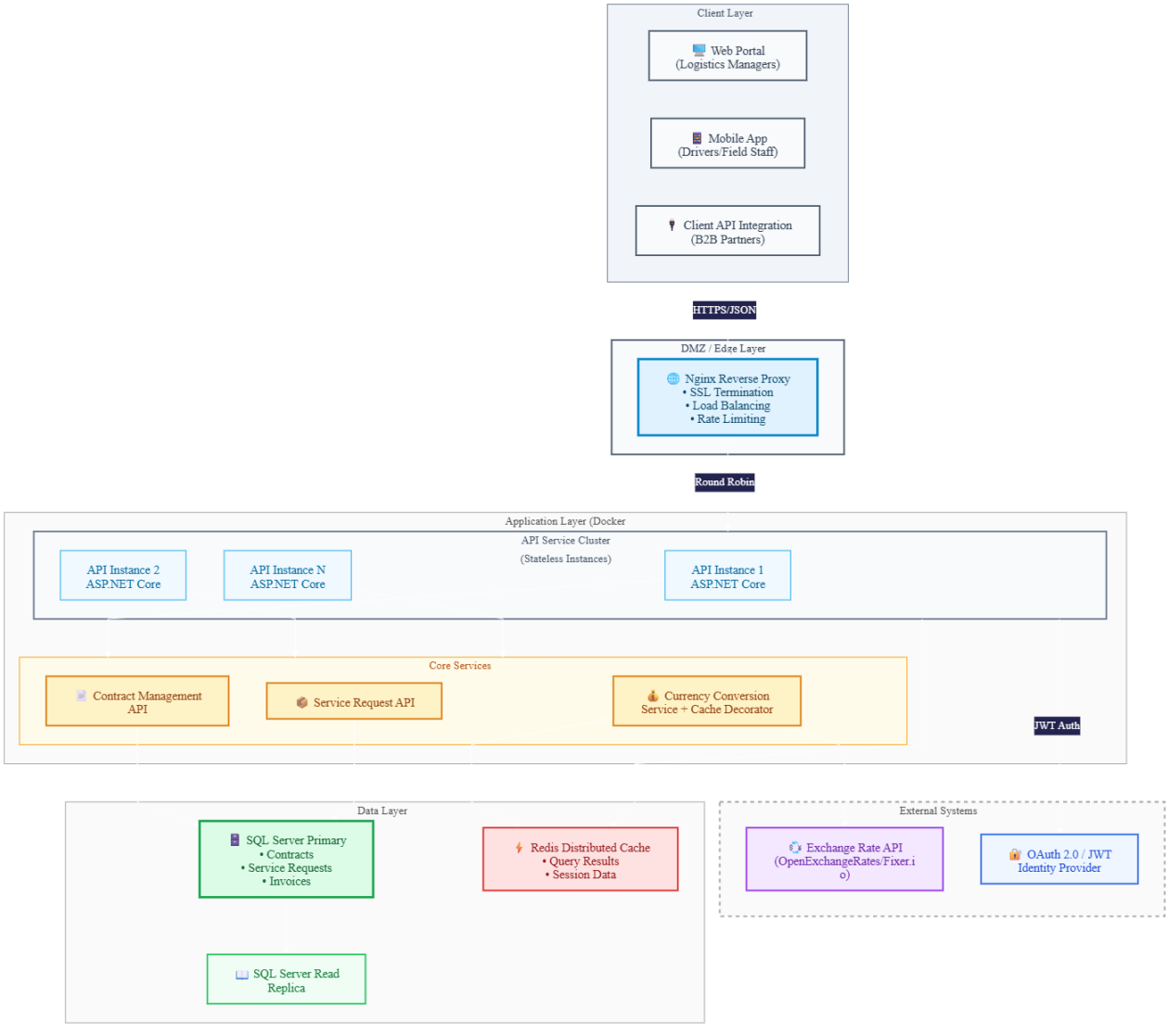
Caching is one of the most effective performance optimisation techniques available to distributed systems, capable of reducing database load by an order of magnitude when applied to high-read-frequency data sets. *(Kleppmann, 2017)*

5.4 Database Scalability

In addition to the three primary strategies above, the database tier must be prepared for increased load. SQL Server supports read replicas, which are synchronised copies of the primary database that can service read queries without affecting the write throughput of the primary instance. Given that the vast majority of GLMS operations are reads (viewing contracts, checking SLAs, browsing service request history), routing read traffic to one or more replicas while directing writes to the primary instance significantly reduces contention and improves overall response times under high load.

Database indexes on the Contract table must be defined on the status column and the ClientId foreign key, as these are the most frequently filtered fields in the application. Without appropriate indexing, a full table scan across 50,000 or more contracts would occur on every filtered query, degrading performance rapidly as data volume grows.

High-Level System Architecture (Modern Enterprise Palette)



6. Conclusion

This report has presented a comprehensive enterprise architecture blueprint for the Global Logistics Management System. TOGAF ADM has been applied across four phases to structure the architectural thinking from vision through to technology specification. Three GoF design patterns, Observer, Strategy, and Decorator, have been selected and justified based on their direct applicability to the GLMS domain. Non-functional requirements of availability and interoperability have been formally defined with concrete implementation strategies, and a multi-layered scalability approach has been outlined that can accommodate rapid large-scale growth without architectural restructuring.

The architecture described in this report forms the design contract that will govern all subsequent development activity. Part 2 will translate this blueprint into a working C# prototype with automated tests, and Part 3 will productionise the system through Docker containerisation and live API integration. The decisions made at this stage, particularly the adoption of stateless API design, interface-driven service boundaries, and container-based deployment, are specifically chosen to ensure that each subsequent phase can proceed without contradiction of the foundational architecture.

References

Abbott, M. L. and Fisher, M. T. (2015) The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise. 2nd edn. Upper Saddle River, NJ: Addison-Wesley.

Bass, L., Clements, P. and Kazman, R. (2021) Software Architecture in Practice. 4th edn. Upper Saddle River, NJ: Addison-Wesley.

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

Kleppmann, M. (2017) Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA: O'Reilly Media.

The Open Group (2022) TOGAF Standard, 10th Edition. Reading: The Open Group. Available at: https://www.opengroup.org/togaf (Accessed: 13 April 2026).